MODULE 6

Collections, filters & HATEOAS

Pagination, action filters, discoverable links

PagedResponse

```
public record PagedResponse<T>
{
  public required IReadOnlyList<T>
    Items { get; init; }
  public required int TotalCount { get; init; }
  public required int Page { get; init; }
  public required int PageSize { get; init; }
  public int TotalPages =>
    (int)Math.Ceiling(TotalCount/(double)PageSize);
  public bool HasPrevious => Page > 1;
  public bool HasNext => Page < TotalPages;
}
```

Scaling your API and making it discoverable

Session goals

By the end of this session, you will:

- Require pagination on collection endpoints with metadata Angular can use
- Cap PageSize to defend against abusive query strings
- Add search/filter/sort in the service with correct LINQ ordering
- Use action filters for cross-cutting controller concerns
- Emit HATEOAS links including conditional enroll when capacity allows

When GET returns tens of thousands

GET /api/courses returns 50,000 records what breaks?

- Server: long JSON serialization seconds on a hot path
- Wire: huge HTTP body tens of MB is normal at that row count
- Client: browser memory and UI thread choke on one giant payload
- Database: wide read + materialization before you ever Skip/Take

Mandatory fix

Pagination + a typed body wrapper (PagedResponse) not “return everything and hope.”

PagedResponse + GetCourses

PagedResponse<T>

```
public record PagedResponse<T>

{
  public required IReadOnlyList<T> Items { get; init; }
  public required int TotalCount { get; init; }
  public required int Page { get; init; }
  public required int PageSize { get; init; }
  public int TotalPages => (int)Math.Ceiling(
    TotalCount / (double)PageSize);
  public bool HasPrevious => Page > 1;

  public bool HasNext => Page < TotalPages;

}
```

- Body wrapper HttpClient reads JSON; no custom header contract
- TotalCount + HasNext / HasPrevious everything Angular needs for pager UI
- M6-Lab-Workbook Exercise 4 same checkpoints as this spine

```
[HttpGet]
public async Task<IActionResult> GetCourses(
  [FromQuery] PagedRequest request,
  CancellationToken ct)
{
  var result = await courseService
    .GetCoursesAsync(request, ct);
  return Ok(result);
}
```

Capping page size

GET /api/courses?pageSize=100000 should the server obey?

PagedRequest

```
public record PagedRequest

{
  private const int MaxPageSize = 50;
  public int Page { get; init; } = 1;
  private int _pageSize = 20;
  public int PageSize {
    get => _pageSize;

    init => _pageSize = value > MaxPageSize

      ? MaxPageSize : value;
  }

  public string? Search { get; init; }

  public string OrderBy { get; init; } = "Title";

}
```

Defense in depth

init silently caps PageSize Postman, curl, or a buggy client never wins a megabyte page.

Filter → count → sort → paginate

Ask the room

Admin search: courses with “web” in the title or code which query string?

Query string

```
GET /api/courses?page=1&pageSize=10&search=web
```

CourseService (TMS)

```
IQueryable<Course> query =
  context.Courses.AsNoTracking();

if (!string.IsNullOrWhiteSpace(request.Search))
  query = query.Where(c =>
    c.Title.Contains(request.Search) ||
    c.Code.Contains(request.Search));

var totalCount = await query.CountAsync(ct);
var items = await query
  .OrderBy(c => c.Title)
  .Skip((request.Page - 1) * request.PageSize)
  .Take(request.PageSize)
  .Select(c => new CourseResponseDto(
    c.Id, c.Title, c.Code, c.MaxCapacity,
    c.Enrollments.Count))
  .ToListAsync(ct);
```

Cue

Filter → count → sort → paginate. Count after Skip/Take lies about totals say the order out loud in lab.

Action filters cross-cutting

Fifty controllers do you paste the same audit logging into every action?

AuditLogFilter

```
public class AuditLogFilter(ILogger<AuditLogFilter> logger): IActionFilter
{
  public void OnActionExecuting(ActionExecutingContext context)
  {
    var route = context.HttpContext.Request.Path;
    var method = context.HttpContext.Request.Method;
    logger.LogInformation(
      "TMS API call: {Method} {Route}", method, route);
  }
  public void OnActionExecuted(
     ActionExecutedContext context)
  {
    var status = context.HttpContext.Response.StatusCode;
    logger.LogInformation(
      "TMS API response: {StatusCode}", status);
  }
}
```

```
builder.Services.AddControllers(
  options =>
  {
    options.Filters.Add<AuditLogFilter>();
  });
```

- Middleware (M4) every HTTP request; action filters only actions that actually run
- Cross-cutting only “is the course full?” stays in EnrollmentService, not in a filter

HATEOAS follow the links(Hypermedia as the Engine of Application State)

Q&A

Backend renames /api/enrollments to /api/courses/{id}/enrollments what breaks if Angular hard-coded URLs?

Course detail JSON (excerpt)

```
{

  "id": "42",
  "title": "Web Development Fundamentals",
  "code": "CSE-101",
  "enrollmentCount": 28,
  "links": [
    { "href": "/api/courses/42",

      "rel": "self", "method": "GET" },

    { "href": "/api/courses/42/enrollments",

      "rel": "enrollments", "method": "GET" },

    { "href": "/api/courses/42",

      "rel": "update", "method": "PUT" },

    { "href": "/api/courses/42",

      "rel": "delete", "method": "DELETE" }
  ]

}
```

- rel names the affordance self, enrollments collection, update, delete
- Client follows href + method server can change routes without a coordinated string replace in Angular

Frontend win

Links are the contract for “what you can do next” not scattered magic strings in TypeScript.

Conditional links enroll when there is room

GetCourseById projection

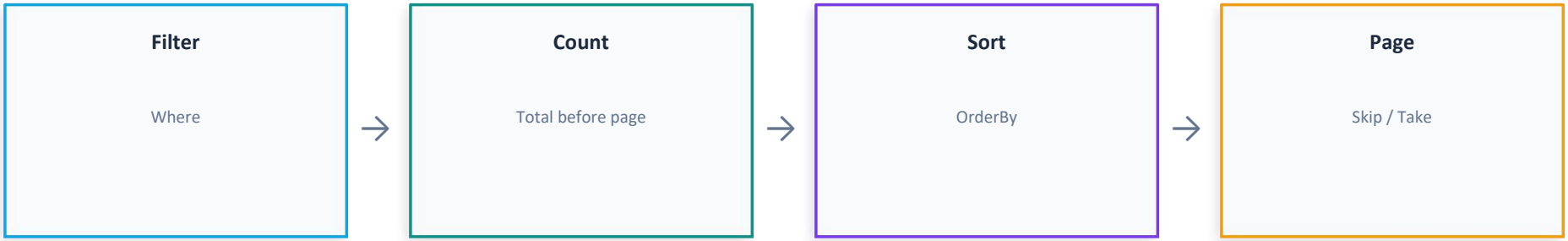
```
var links = new List<LinkDto>

{

  new($"/api/courses/{id}", "self", "GET"),

  new($"/api/courses/{id}", "update", "PUT"),

  new($"/api/courses/{id}", "delete", "DELETE")

};
```

- Angular enables Enroll only when an enroll link is present no duplicated “is the course full?” in the browser
- Backend remains the single source of truth for which actions exist for this course state

Interview line

This is where HATEOAS pays rent business state drives the link array, not parallel TS logic.

Blueprint collection pipeline



Lab brief Session 3

- Exercise 4: PagedRequest / PagedResponse, search, cap, correct TotalCount
- Exercise 5: LinkDto, CourseDetailDto, HATEOAS on GetCourseById
- If stuck: counting after Skip/Take, or missing PageSize cap

Session summary

Topic	Takeaway
Pagination	Mandatory for collections; PagedResponse metadata for the pager UI
Caps	Never trust client pageSize init caps in PagedRequest
LINQ order	Filter → count → sort → paginate count before Skip/Take
Filters	Action filters for audit/timing not enrollment or domain rules
HATEOAS	rel + href + method in JSON clients follow links instead of string-built URLs
Conditional links	Add enroll POST only when EnrollmentCount < MaxCapacity Angular mirrors the array

What this looks like in an interview

They want to hear: mandatory pagination, capped page size, correct total count, and why links beat hard-coded URLs for evolving APIs.